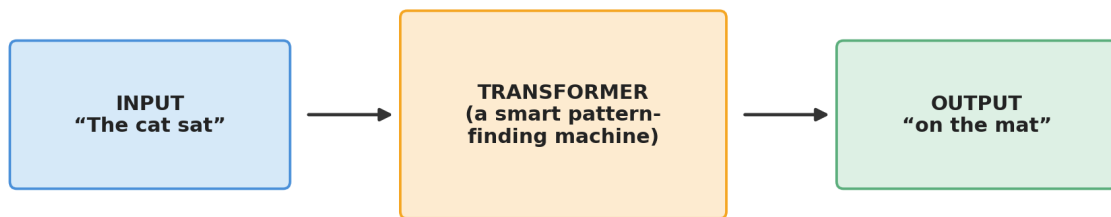


Transformers & Vision Transformers

A Beginner-Friendly Illustrated Guide for Students



It reads ALL the words at once and figures out how they relate to each other

What you will learn

- What a Transformer is and why it changed AI forever
- How **self-attention** works — explained with pictures, not scary math
- The full Transformer architecture, block by block
- How the **Vision Transformer (ViT)** applies the same idea to images
- How ViT compares to CNNs, plus a glossary and quiz at the end

No prior deep-learning experience needed — just curiosity!

1. Why Should You Care About Transformers?

In 2017, a research paper with the catchy title “*Attention Is All You Need*” introduced a new neural network design called the **Transformer**. It looked like just another architecture at the time — but it turned out to be one of the most important inventions in the history of artificial intelligence.

Almost every famous AI system you have heard of is built on Transformers: **ChatGPT**, **Claude**, **Gemini**, **BERT** (which powers Google Search), translation apps, code assistants, and even many image and video models. Learning how a Transformer works means understanding the engine inside modern AI.

The Problem Transformers Solved

Before Transformers, language models used **Recurrent Neural Networks (RNNs)**. An RNN reads a sentence one word at a time, left to right, like reading through a keyhole. This causes two big problems:

- 1. It's slow.** You cannot process word 10 until you have finished words 1–9, so you can't use parallel computing hardware (GPUs) efficiently.
- 2. It forgets.** By the time an RNN reaches the end of a long paragraph, information from the beginning has faded — like a game of telephone.

The Transformer's big idea: **throw away the one-word-at-a-time reading entirely**. Instead, look at all the words *at the same time*, and let every word directly “talk to” every other word. The mechanism that makes this possible is called **attention**.

Analogy: Group photo vs. slideshow

An RNN understands a scene by watching a slideshow, one picture at a time, trying to remember what it saw. A Transformer looks at a single group photo where everything is visible at once — it can instantly see who is standing next to whom.

2. Step Zero: Turning Words into Numbers

Neural networks can only do math, so text must first become numbers. This happens in two steps:

Tokenization: The text is chopped into small pieces called **tokens** (roughly words or word fragments). “The cat sat” becomes three tokens.

Embedding: Each token is mapped to a long list of numbers called an **embedding vector** (often 512 or more numbers). Think of it as the word's coordinates in a giant “meaning space”: words with similar meanings (like *cat* and *kitten*) end up close together.

Step 1: Words → Tokens → Number Vectors (Embeddings)

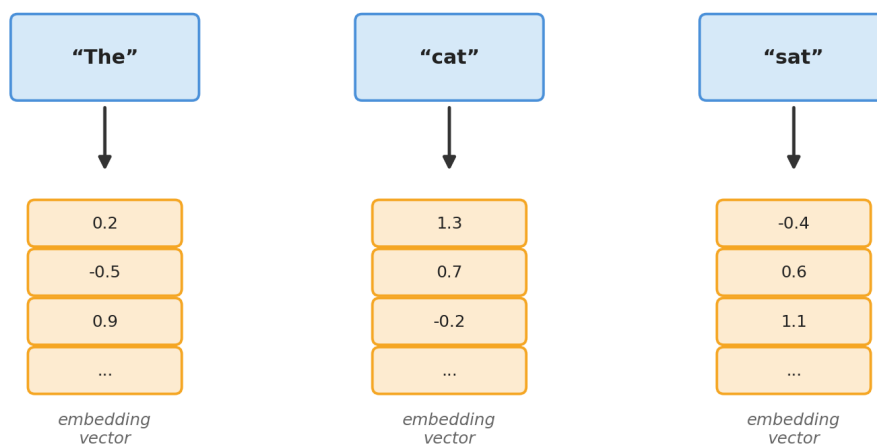


Figure 1 — Every token becomes a vector of numbers that captures its meaning.

3. The Star of the Show: Self-Attention

Read this sentence: “*The animal didn't cross the street because **it** was tired.*” What does “it” refer to — the animal or the street? You instantly know it's the animal. You did that by *relating words to each other*. Self-attention lets a neural network do exactly the same thing.

Self-Attention: the word “it” looks at every other word

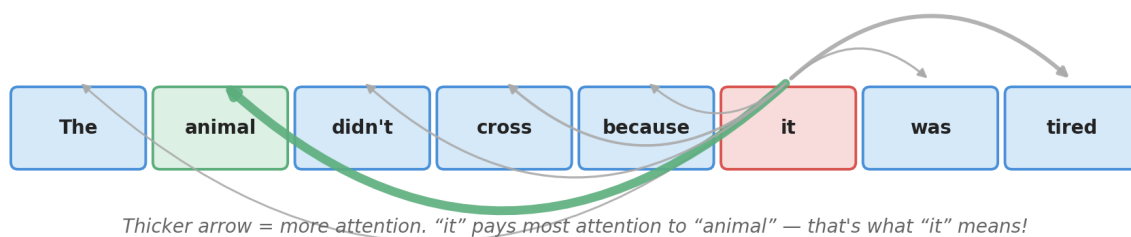


Figure 2 — Each word computes how much “attention” to pay to every other word.

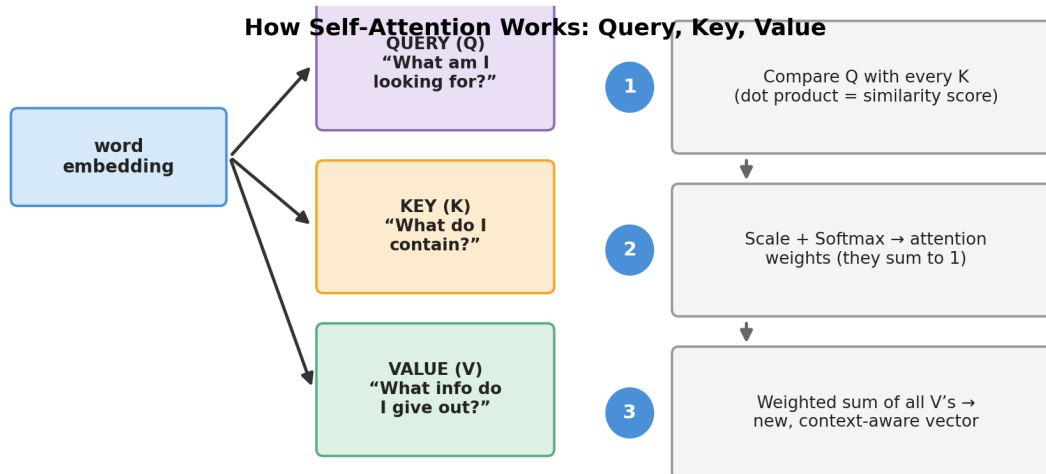
In self-attention, **every word looks at every other word** and asks: “How relevant are you to me?” The answers become **attention weights** — percentages that sum to 100%. The word then updates its own representation by blending in information from the words it attends to most. After this step, the vector for “it” literally contains information borrowed from “animal”.

How It Works: Queries, Keys and Values

Every word's embedding is transformed into three different vectors, each with its own job. A great analogy is searching a video website:

- **Query (Q)** — what you type in the search bar: “what am I looking for?”
- **Key (K)** — the title/tags of each video: “what do I contain?”
- **Value (V)** — the video itself: “the actual content you get back.”

Your query is compared against every key. Good matches score high; those scores decide how much of each value gets mixed into the result.



$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Figure 3 — The three-step attention recipe: compare Q with K , turn scores into weights, blend the V s.

The famous formula — decoded piece by piece

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

- QK^T — compare every query with every key (dot products = similarity scores)
- $\div \sqrt{d_k}$ — shrink the scores so they don't get too extreme (keeps training stable)
- **softmax** — convert scores into positive weights that add up to 1 (i.e., percentages)
- $\cdot V$ — take a weighted average of the value vectors using those percentages

That's it. The scariest formula in AI is really just: *score, normalize, average*.

Attention Maps: Seeing What the Model Sees

We can visualize attention weights as a grid. Each row shows one word's attention budget spread over all the words in the sentence (each row sums to 1.0). Darker cells mean stronger attention.

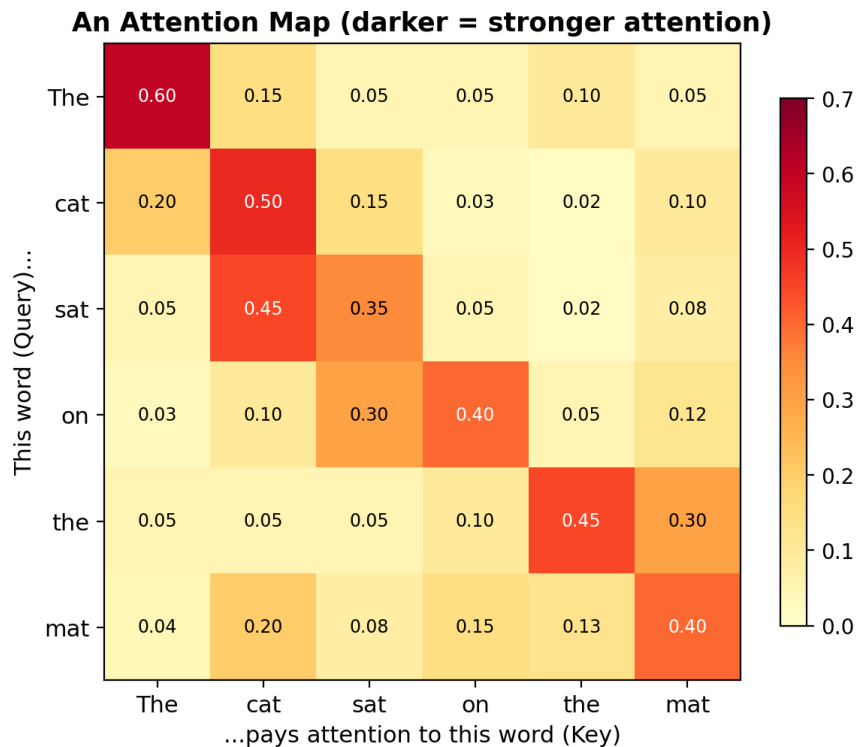


Figure 4 — A toy attention map. Notice how “sat” attends strongly to “cat” (who is sitting?).

Multi-Head Attention: Many Points of View

One attention pass captures one type of relationship. But language has many kinds of relationships at once: grammar (subject–verb), meaning (pronoun–noun), position (nearby words), and more. So Transformers run **several attention “heads” in parallel**, each with its own Q, K, V transformations. Each head learns to specialize in a different pattern, and their outputs are concatenated and mixed together.

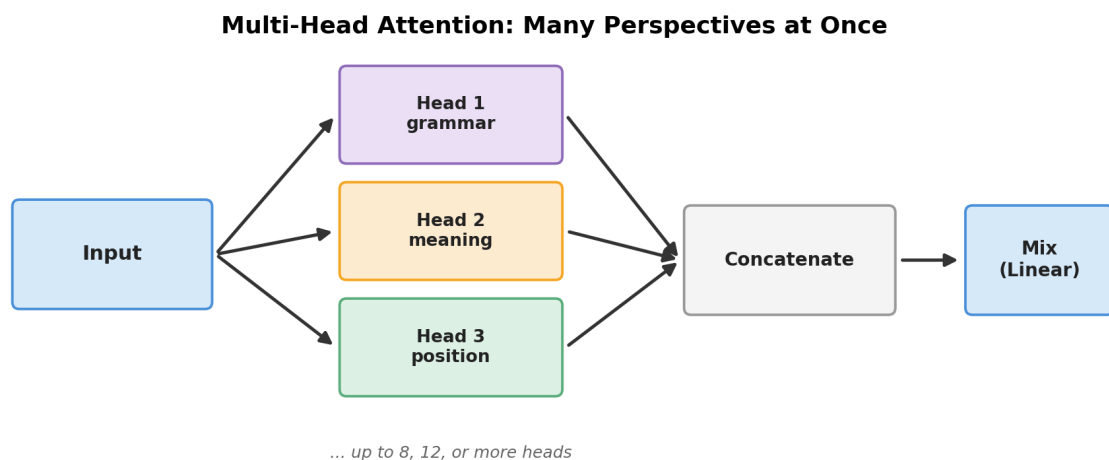


Figure 5 — Multiple heads = multiple experts reading the same sentence for different clues.

Analogy: A panel of specialists

Imagine giving one essay to a grammar teacher, a poet, and a fact-checker. Each notices different things. Combining their notes gives a far richer understanding than any single reader — that's multi-head attention.

4. Wait — How Does It Know Word Order?

Because attention looks at all words simultaneously, it is order-blind: “dog bites man” and “man bites dog” would look identical! The fix is **positional encoding**: before the first layer, we add a unique “position signature” to each word's embedding. The original paper used sine and cosine waves of different frequencies — every position gets its own unmistakable wave pattern, a bit like a barcode.

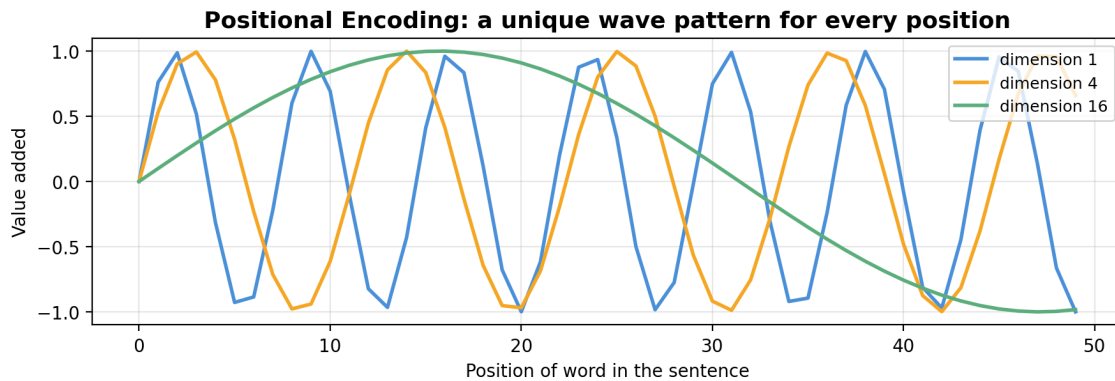


Figure 6 — Sinusoidal positional encodings: each position in the sentence receives a unique pattern.

5. Assembling the Full Transformer

Now we can put all the pieces together. The original Transformer has two halves:

- The **Encoder** (left) reads and *understands* the input sentence.
- The **Decoder** (right) *generates* the output one token at a time — for example, a translation — while peeking at the encoder's understanding through **cross-attention**.

The Full Transformer (from “Attention Is All You Need”, 2017)

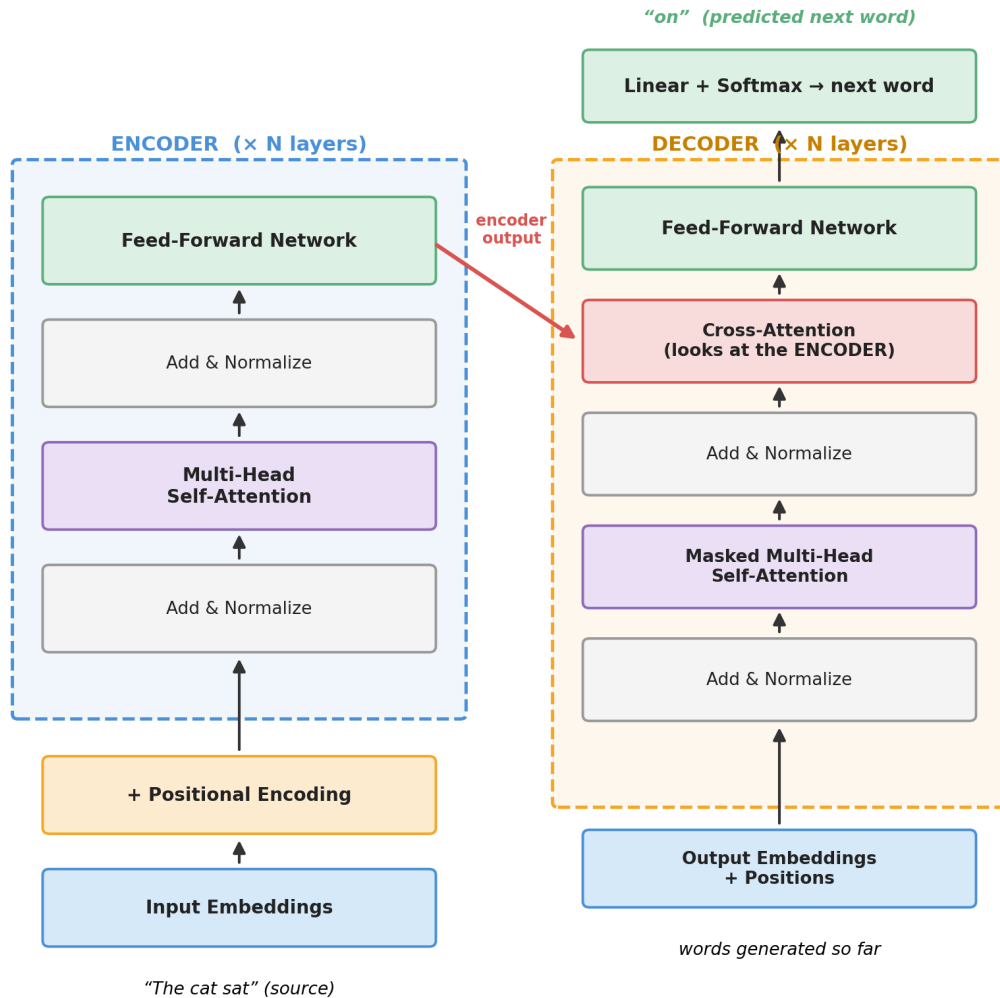


Figure 7 — The complete Transformer. Each grey “Add & Normalize” step includes a shortcut (residual) connection that keeps training stable.

A few supporting parts appear in every block:

- **Feed-Forward Network:** a small two-layer neural net applied to each word separately — it gives the model extra “thinking capacity” after attention has gathered context.
- **Residual connections (Add):** shortcuts that add each block's input to its output, so information and gradients can flow easily through deep stacks.
- **Layer Normalization:** keeps the numbers in a healthy range so training doesn't blow up.
- **Masked self-attention (decoder only):** when generating, the model is forbidden from peeking at future words — no cheating on the answer!

Fun fact: GPT vs BERT

Modern models often use only half of this diagram. **BERT** is a stack of *encoders* (great at understanding). **GPT-style models** (ChatGPT, Claude) are stacks of *decoders* (great at generating text). Same Lego bricks, different buildings!

6. The Vision Transformer (ViT): Same Trick, Now with Pictures

For a decade, **Convolutional Neural Networks (CNNs)** ruled computer vision. Then in 2020, researchers at Google asked a bold question: “*What if we barely change the Transformer at all, and just feed it images instead of words?*” The result was the **Vision Transformer (ViT)** — and it matched or beat the best CNNs on major benchmarks.

The Key Insight: An Image Is a Sentence of Patches

A Transformer eats a *sequence* of vectors. Words form a natural sequence — but an image is a 2D grid of pixels. The ViT solution is beautifully simple: **cut the image into small square patches** (usually 16×16 pixels), flatten each patch into a vector, and line them up in a row. Now each patch plays the role of a “word”, and the whole image is a “sentence”.

Vision Transformer Idea: An Image Is Just a “Sentence of Patches”

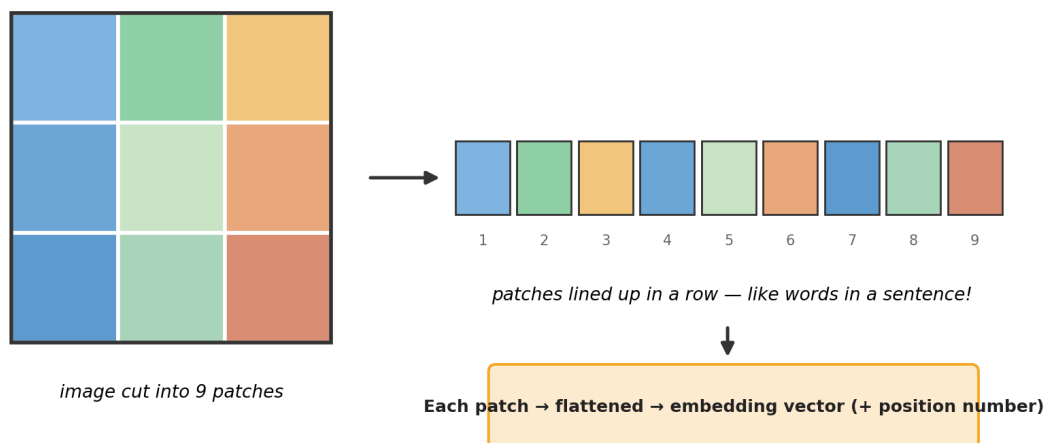


Figure 8 — Chop, flatten, embed: the image becomes a sequence the Transformer already knows how to read.

A 224×224 image cut into 16×16 patches gives $14 \times 14 = 196$ patches — a “sentence” of 196 visual words. Each patch is flattened ($16 \times 16 \times 3$ colors = 768 numbers) and passed through a linear layer to create its embedding, and a **position embedding** is added so the model knows where each patch came from in the grid.

The [CLS] Token: A Seat for the Summary

ViT adds one extra, special learnable token called **[CLS]** (“classification”) to the front of the patch sequence. As it passes through the layers, this token attends to all the patches and gradually collects a summary of the whole image. At the end, we read the final [CLS] vector, pass it through a small classifier (MLP head), and out comes the answer: “cat”, “dog”, “airplane”...

Vision Transformer (ViT) — Full Pipeline

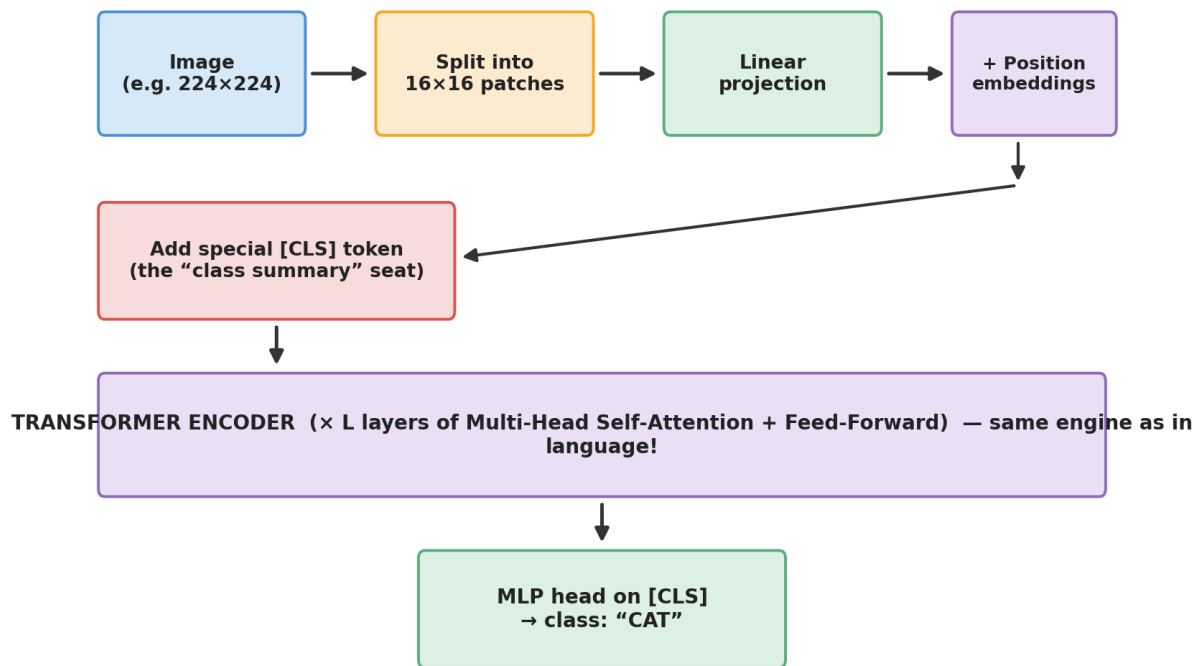


Figure 9 — The full ViT pipeline. The encoder in the middle is the SAME architecture used for language.

ViT vs CNN: What's Actually Different?

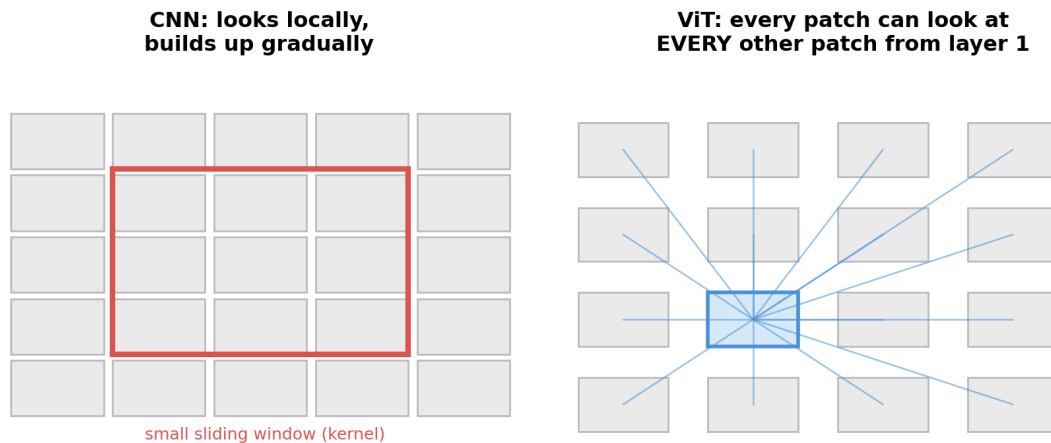


Figure 10 — A CNN's kernel sees a small neighborhood; a ViT patch can attend to the entire image from the very first layer.

Aspect	CNN	Vision Transformer
How it looks at images	Small sliding windows; local first, global later (layer by layer)	Global self-attention from layer 1 — any patch can talk to any patch
Built-in assumptions	Strong: nearby pixels matter, patterns repeat anywhere (translation invariance)	Very few — it must learn spatial structure from data
Data appetite	Works well even on smaller datasets	Needs LOTS of data (or clever pre-training) to shine
Scaling up	Gains flatten out at very large sizes	Keeps improving with more data and compute
Interpretability	Feature maps	Attention maps show exactly where the model “looked”

Why does ViT need so much data?

A CNN is born knowing that “nearby pixels belong together”. A ViT starts with almost no assumptions about images — it must discover concepts like edges, textures and locality entirely from examples. With enough data (millions of images), that freedom becomes a superpower: it can learn relationships a CNN's rigid structure would miss. That's why ViTs are usually **pre-trained** on huge datasets and then fine-tuned.

7. The Big Picture Recap

1. Turn the input into a sequence of vectors — word embeddings for text, patch embeddings for images.
2. Add positional information so order/layout isn't lost.
3. Pass the sequence through stacked blocks of **multi-head self-attention** (gather context) and **feed-forward networks** (process it), glued together with residual connections and normalization.
4. Read the output: predict the next word (language) or classify via the [CLS] token (vision).

The deepest lesson: **the same architecture works for text, images, audio, protein sequences and more**. If you can turn your data into a sequence of vectors, a Transformer can learn from it.

Mini Glossary

Token	A small chunk of input — a word/word-piece in text, or a patch in ViT.
Embedding	A vector of numbers representing a token's meaning.
Self-attention	Mechanism letting every token weigh and borrow information from every other token.
Query / Key / Value	Three roles each token plays: what I seek / what I offer / what I hand over.
Softmax	Turns raw scores into positive weights that sum to 1 (percentages).
Attention head	One independent attention pattern; many run in parallel.
Positional encoding	Extra signal added so the model knows each token's position.
Residual connection	A shortcut adding a block's input to its output; helps deep training.
Layer normalization	Rescales values to keep training numerically stable.
Encoder / Decoder	Understanding stack / generating stack of the original Transformer.
[CLS] token	Special token in ViT/BERT that gathers a whole-input summary for classification.
Patch	A small square of an image (e.g. 16×16 pixels) treated like a word.

8. Check Your Understanding (Quiz)

Try these before looking at the answers at the bottom of the page!

1. What are the two main problems with RNNs that Transformers solved?
2. In the Query/Key/Value analogy, which one represents “what you are searching for”?
3. Why do Transformers need positional encodings at all?
4. What does the softmax function do to the attention scores?
5. Why does a Transformer use multiple attention heads instead of just one?
6. How does a Vision Transformer turn a 2D image into a sequence?
7. What is the job of the [CLS] token in ViT?
8. Give one advantage of a CNN over a ViT, and one advantage of a ViT over a CNN.

Answers

1. Sequential (slow, no parallelism) and forgetful over long distances. **2.** The Query. **3.** Self-attention treats input as an unordered set — without positions, word order is invisible. **4.** Converts scores into positive weights that sum to 1. **5.** Each head can specialize in a different type of relationship (grammar, meaning, position...). **6.** It cuts the image into fixed-size patches, flattens each one and embeds it — patches become “words”. **7.** It attends to all patches and accumulates a summary of the whole image, used for classification. **8.** CNN: works well with less data thanks to built-in image assumptions. ViT: sees the whole image from layer 1 and scales better with huge datasets.

Where to Go Next

- **“Attention Is All You Need”** (Vaswani et al., 2017) — the original Transformer paper.
- **“An Image is Worth 16×16 Words”** (Dosovitskiy et al., 2020) — the ViT paper.
- **The Illustrated Transformer** by Jay Alammar — a classic visual blog walkthrough.
- Try coding one! Hugging Face and PyTorch tutorials let you run a pre-trained ViT in ~10 lines of code.